Les interactions avec le dépôt distant

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Travailler avec un repo distant **3**

A. Configuration de GitHub 3

B. Brancher un dépôt distant 7

C. Exercice : Quiz 9

III. L'interaction entre le local et le distant **10**

A. L'interaction entre le local et le distant 10

B. Exercice : Quiz 13

IV. Essentiel **14**

V. Auto-évaluation **14**

A. Exercice 14

B. Test 16

Solutions des exercices **16**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : GitBash / Terminal

Prérequis : avoir installé Git sur sa machine

Contexte

Au fur et à mesure de votre vie de développeur, vous travaillerez rarement seul, vous aurez toujours des collaborateurs sur vos projets. Le travail sera réparti entre vous et vous allez coder sur le même projet, en même temps. Par conséquent, vous aurez besoin d'avoir des outils qui vous permettent de fusionner votre travail, sans perdre vos modifications. Pour cela, il est d'usage d'utiliser des outils de contrôle de version, c'est-à-dire des outils qui permettent de centraliser le code afin que tous les membres de l'équipe puissent y accéder.

Ces outils de contrôle de version facilitent la collaboration entre les développeurs en permettant le suivi des modifications apportées au code source d'un projet. L'un des systèmes de contrôle de version les plus populaires est Git, qui offre une grande flexibilité et des fonctionnalités avancées.

L'utilisation de Git permet à chaque membre de l'équipe de travailler sur une copie locale du code source, apportant des modifications et les sauvegardant dans des « *branches* » distinctes. Ces branches sont essentiellement des copies temporaires du code, permettant aux développeurs de travailler sur des fonctionnalités spécifiques ou de corriger des bogues sans affecter le code principal du projet.

II. Travailler avec un repo distant

A. Configuration de GitHub

Fondamental **Repository distant (dépôt ou référentiel distant)**

Un repository distant est une version du dépôt Git hébergée sur un serveur distant. Il s'agit d'une copie centrale du code source qui peut être partagée et accessible par plusieurs développeurs. Finalement, c'est une copie du repository que vous avez en local sur votre machine, mais hébergée sur une plateforme.

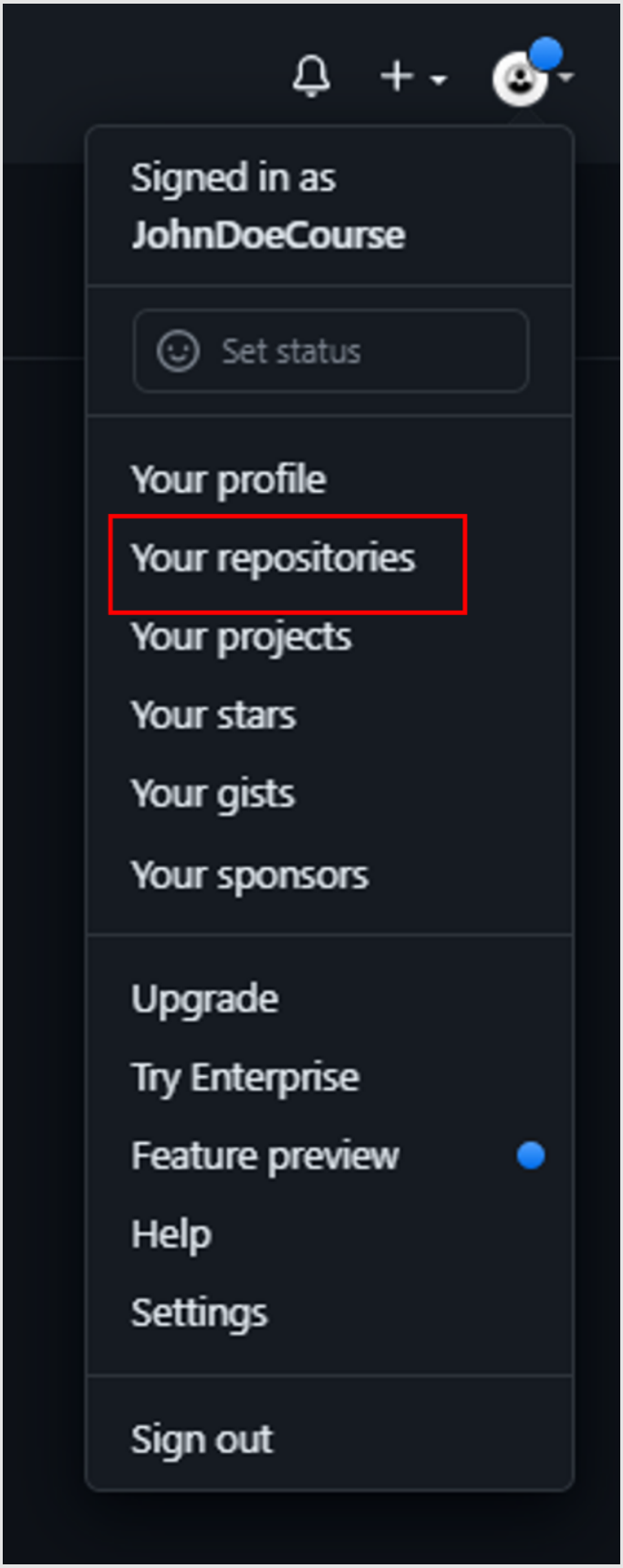
Les dépôts distants sont souvent hébergés sur des plateformes comme GitHub, Bitbucket, GitLab, ou encore sur des serveurs d'entreprise. Ces plateformes offrent des fonctionnalités supplémentaires, comme la gestion des problèmes, des demandes de fusion (*pull requests*), la gestion des autorisations d'accès, la revue de code, etc.

Méthode **Créer son compte GitHub**

GitHub est une plateforme en ligne d'hébergement de code source. Pour pouvoir l'utiliser, il faut créer un compte. Vous pouvez créer votre compte comme sur n'importe quel site. GitHub vous propose différents plans, dont certains gratuits et d'autres payants. Avec un compte gratuit, vous aurez accès à un bon nombre de fonctionnalités.

Méthode **La gestion des dépôts sur GitHub**

En cliquant sur votre avatar en haut à droite de la page, vous avez accès à différentes options de GitHub. Vous pouvez cliquer sur « ***Your repositories*** » pour accéder à la liste de vos dépôts.



Tous droits réservés à STUDI - Reproduction interdite

Image : Repositories Github

Vous êtes maintenant sur la liste de vos dépôts. Vous trouverez ici tous vos dépôts distants, c'est-à-dire tous vos dépôts présents sur GitHub. Si vous voulez en créer un nouveau, vous pouvez cliquer sur le bouton « ***new*** ».

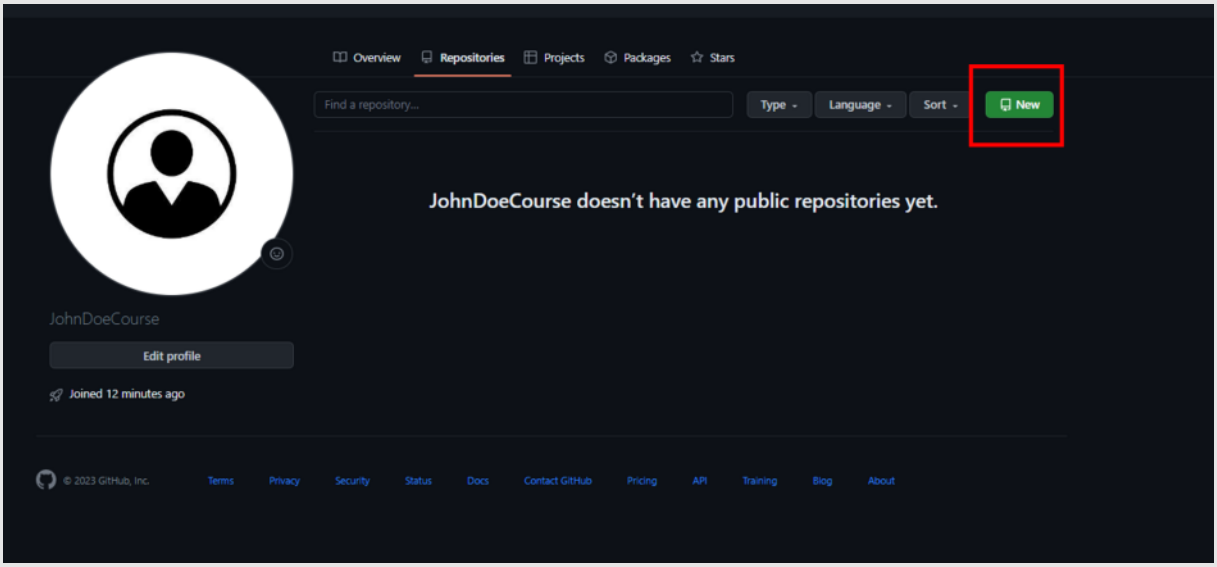


Image : New repo button

La configuration de son repository

Il faut maintenant configurer votre repository. Vous pouvez remplir ces différentes options :

- Repository Name (nom du dépôt) : c'est le nom que vous donnez à votre dépôt. Choisissez un nom descriptif et significatif qui représente le projet ou le contenu du dépôt.
- Description : ce champ vous permet de fournir une brève description de votre dépôt. Décrivez en quelques mots le but du projet ou les fonctionnalités principales du dépôt. Ce champ est facultatif.
- Public / Private (public / privé) : vous pouvez choisir de rendre votre dépôt public ou privé. Les dépôts publics sont visibles par tous les utilisateurs de GitHub, tandis que les dépôts privés sont accessibles uniquement aux personnes que vous invitez explicitement à y accéder.
- Initialize this repository with a README (initialiser ce dépôt avec un fichier README) : cochez cette case si vous souhaitez créer un fichier README lors de la création du dépôt. Un fichier README est souvent utilisé pour fournir des informations sur le projet, des instructions d'installation ou des exemples d'utilisation.
- .gitignore : si vous cochez cette case, vous pouvez choisir un modèle de fichier .gitignore préconfiguré pour exclure certains fichiers ou dossiers spécifiques du suivi de version Git. Cela peut inclure des fichiers temporaires, des fichiers générés ou des fichiers sensibles que vous ne souhaitez pas inclure dans votre dépôt. Si vous ne savez pas quoi choisir, vous pouvez laisser à « *none* ».
- License (licence) : si vous cochez cette case, vous pouvez choisir une licence logicielle pour votre projet. Les licences définissent les conditions d'utilisation, de modification et de distribution de votre code. Sélectionnez la licence appropriée pour votre projet ou choisissez « *none* » si vous ne souhaitez pas inclure de licence. Pour vos projets étudiants, vous n'aurez généralement pas besoin de cette option.

Tous droits réservés à STUDI - Reproduction interdite

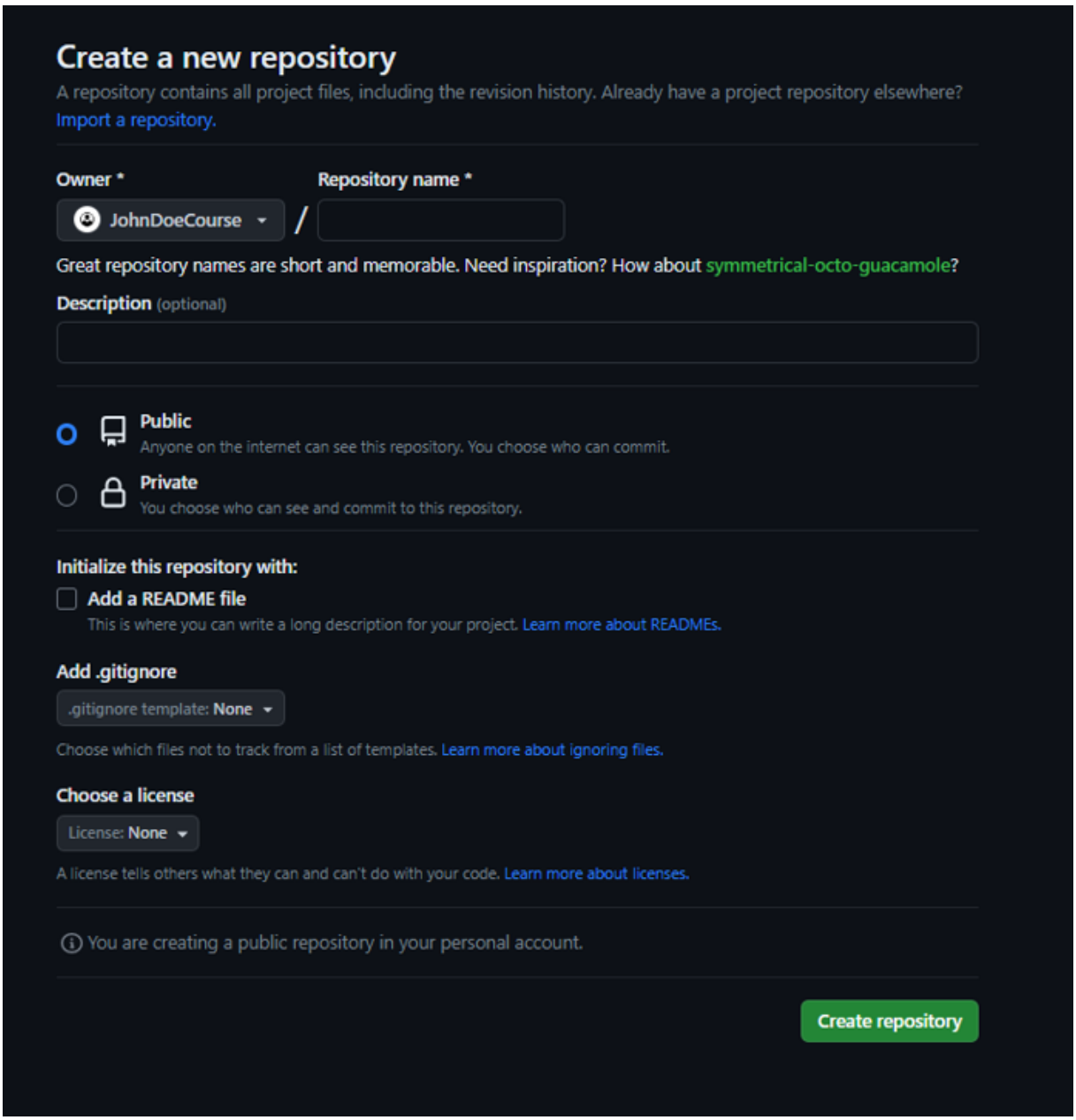


Image : New repo config

Vous pouvez maintenant créer votre dépôt en cliquant sur le bouton « ***Create repository*** ». Votre repository distant est créé ! Pour pouvoir travailler avec, il vous reste à brancher un repository local à ce nouveau repository.

Complément **Les différents onglets**

Depuis la page de détail de votre repository, vous avez accès à différents onglets liés à votre repository. Pour le moment, nous n'allons pas nous servir de beaucoup d'entre eux, mais à terme, ils pourront vous être très utiles. Voici leurs descriptions :

1. **Code** : cet onglet affiche le contenu de votre dépôt, y compris les fichiers, les dossiers et l'historique des commits. Vous pouvez visualiser, éditer et créer de nouveaux fichiers directement sur GitHub, ainsi que parcourir les différentes branches et tags de votre dépôt.

Tous droits réservés à STUDI - Reproduction interdite

2. **Issues** (problèmes) : cet onglet est utilisé pour gérer les problèmes (*issues*) liés à votre projet. Vous pouvez créer de nouveaux problèmes, suivre leur état, discuter des problèmes avec les collaborateurs et attribuer des tâches à des membres de l'équipe.
3. **Pull requests** (demandes de fusion) : cet onglet est utilisé pour gérer les demandes de fusion (*pull requests*). Les demandes de fusion permettent à d'autres contributeurs de proposer des modifications à votre dépôt et de demander leur intégration. Vous pouvez examiner, discuter et fusionner ces demandes de fusion.
4. **Actions** : cet onglet vous permet de configurer et de visualiser les automatisations et les actions associées à votre dépôt. On y trouve généralement des actions de tests, le déploiement et les notifications.
5. **Projects** (projets) : cet onglet vous permet de créer et de gérer des tableaux Kanban pour organiser vos tâches, suivre l'état du projet et collaborer avec votre équipe. C'est un onglet de gestion de projet.
6. **Wiki** : cet onglet est utilisé pour créer et éditer une documentation spécifique à votre projet. Vous pouvez fournir des informations supplémentaires sur l'utilisation, l'installation, les guides de contribution, etc.
7. **Security** (sécurité) : cet onglet fournit des informations sur les alertes de sécurité détectées dans votre code source. Vous pouvez prendre des mesures pour remédier aux problèmes de sécurité identifiés.
8. **Insights** (informations) : cet onglet vous fournit des informations et des statistiques sur votre dépôt, telles que les activités de contribution, les graphiques d'utilisation du code, les statistiques des contributeurs et d'autres informations utiles pour évaluer l'état et les performances de votre projet.
9. **Settings** (paramètres) : vous aurez accès aux paramètres de votre repo, vous pourrez le renommer, le supprimer, changer son accessibilité, etc.

B. Brancher un dépôt distant

Maintenant que le dépôt distant est créé, il faut créer le dépôt local et le lier à notre dépôt distant. Nous avons besoin pour commencer de l'URL de notre dépôt distant, que nous pouvons trouver sur GitHub avec le bouton « **<> *Code*** ». Cette URL sera l'adresse sur laquelle nous pourrons déposer ou récupérer notre code. Dans l'exemple qui suit, pensez à remplacer l'URL « https://github.com/JohnDoeCourse/MyFirstRepo.git » par l'URL de votre projet.

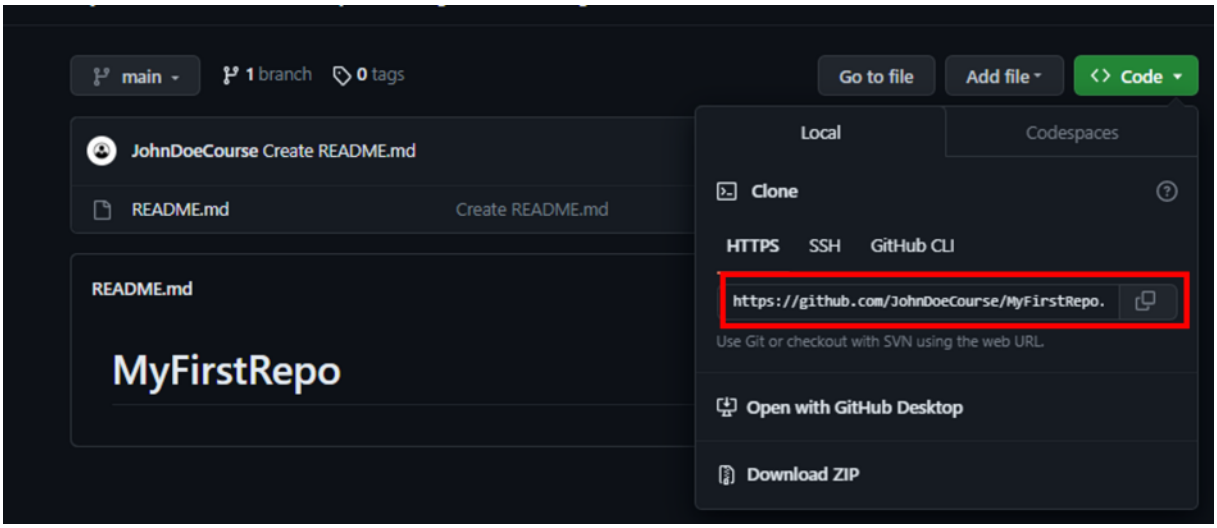


Image : Url repo

À ce moment, 2 options s'offrent à vous.

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Si vous avez du code en local que vous voulez envoyer sur le nouveau dépôt, vous allez pouvoir utiliser `git push`.

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/JohnDoeCourse/MyFirstRepo.git
git push -u origin main
```

Voici quelques explications :

- `'git init'` : cette commande initialise un nouveau dépôt Git dans le répertoire actuel. Elle crée un dossier caché nommé « *.git* » qui stocke tous les fichiers et l'historique de votre projet Git. Vous devez exécuter cette commande une seule fois au début de la création de votre dépôt.
- `'git add .'` : cette commande ajoute toutes les modifications et les nouveaux fichiers de votre répertoire de travail à l'index de Git. L'index est une zone de préparation où vous spécifiez les changements à inclure dans votre prochain commit. En utilisant cette commande, vous préparez tous les fichiers modifiés pour être enregistrés dans l'historique.
- `'git commit -m "first commit"'` : avec cette commande, vous créez un nouveau commit qui enregistre les modifications ajoutées à l'index. Le message entre guillemets, tel que « *first commit* », décrit brièvement les changements effectués dans ce commit. Vous pouvez personnaliser le message pour décrire les modifications spécifiques que vous avez apportées.
- `'git branch -M main'` : cette commande renomme la branche principale (habituellement « *master* ») en « *main* ». C'est une pratique courante pour adopter une terminologie plus inclusive dans Git.
- `'git remote add origin https://github.com/JohnDoeCourse/MyFirstRepo.git'` : utilisez cette commande pour ajouter une référence à un dépôt distant appelé « *origin* ». L'URL spécifiée correspond à l'emplacement du dépôt distant sur GitHub. En utilisant « *origin* », vous pouvez facilement faire référence à ce dépôt distant dans d'autres commandes Git.
- `'git push -u origin main'` : cette commande envoie vos commits locaux vers le dépôt distant spécifié par « *origin* ». L'option -u définit la branche locale « *main* » comme branche de suivi pour la branche « *main* » du dépôt distant. Cela vous permet d'utiliser simplement `git push` à l'avenir pour pousser vos modifications vers le dépôt distant.

Méthode

Si vous n'avez pas encore commencé le code de votre projet, vous allez pouvoir utiliser `git clone`.

```
git clone https://github.com/JohnDoeCourse/MyFirstRepo.git
git branch -M main
git push -u origin main
```

Voici quelques explications :

- `'git clone https://github.com/JohnDoeCourse/MyFirstRepo.git'` : cette commande permet de cloner (ou brancher) un dépôt distant sur votre machine locale. L'URL spécifiée correspond à l'emplacement du dépôt distant sur GitHub. Lorsque vous exécutez cette commande, Git télécharge tous les fichiers du dépôt distant et crée une copie locale complète du dépôt sur votre machine.
- `'git branch -M main'` : cette commande renomme la branche principale (par défaut « *master* » avant Git 2.28) en « *main* ». L'option -M est utilisée pour forcer le renommage de la branche, même si une branche du même nom existe déjà.

Tous droits réservés à STUDI - Reproduction interdite

- `'git push -u origin main'` : cette commande envoie les commits locaux de la branche « *main* » vers le dépôt distant spécifié par « *origin* ». L'option -u définit la branche locale « *main* » comme branche de suivi pour la branche « *main* » du dépôt distant. Cela permet de simplifier les futures commandes `git push` sans avoir à spécifier les arguments de branche.

Remarque **Configurer Git pour être lié à votre compte GitHub**

Pour pouvoir interagir avec votre dépôt distant, il faut lier votre Git local à votre compte GitHub.

Pour configurer Git pour GitHub, ouvrez un terminal et entrez les commandes suivantes :

```
git config --global user.name "Nom d'utilisateur GitHub"
git config --global user.email "Adresse email GitHub"
```

Remplacez « *Nom d'utilisateur GitHub* » par votre nom d'utilisateur GitHub et « *Adresse email GitHub* » par l'adresse e-mail associée à votre compte GitHub.

La prochaine fois que vous clonez une URL HTTPS qui nécessite une authentification, Git vous demandera de vous connecter en utilisant une fenêtre de navigateur. Il se peut que vous soyez d'abord invité à autoriser une application OAuth. Si votre compte ou votre organisation requiert une authentification à 2 facteurs, vous devrez également compléter l'authentification à 2 facteurs.

Une fois que vous vous êtes authentifié avec succès, vos identifiants sont stockés dans le gestionnaire de mots de passe de Windows et seront utilisés chaque fois que vous clonez une URL HTTPS. Git ne vous demandera pas de saisir à nouveau vos identifiants en ligne de commande, à moins que vous ne les modifiiez.

(Voir : Caching your GitHub credentials in Git)[1]

C. Exercice : Quiz

[solution n°1 p.17]

Question 1

Qu'est-ce qu'un repository distant (dépôt distant) sur GitHub ?

- ○ Une version du dépôt Git hébergée sur un serveur distant
- ○ Une copie de sauvegarde du dépôt local
- ○ Un dépôt utilisé uniquement pour le développement local

Question 2

Quelles sont les principales plateformes d'hébergement pour les repositories distants ?

- ○ Azure, AWS, Google Cloud
- ○ GitHub, GitLab, Bitbucket
- ○ WordPress, Joomla, Drupal

Question 3

Comment crée-t-on un compte sur GitHub ?

- ○ En se rendant sur le site et en remplissant un formulaire d'inscription
- ○ En téléchargeant une application dédiée sur son ordinateur
- ○ En utilisant une commande spécifique dans le terminal

Question 4

1 https://docs.github.com/en/get-started/getting-started-with-git/caching-your-github-credentials-in-git

Tous droits réservés à STUDI - Reproduction interdite

Où peut-on trouver la liste de ses dépôts sur GitHub ?

- ☐ En utilisant la commande `git list` dans le terminal
- ☐ En se rendant sur la page d'accueil de son profil
- ☐ En cliquant sur son avatar en haut à droite de la page

Question 5

Quels sont les onglets disponibles sur la page de détail d'un repository sur GitHub ?

- ○ Files, Commits, Branches, Merge requests, Issues, Actions, Settings
- ○ Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings
- ○ Overview, Files, History, Issues, Settings

III. L'interaction entre le local et le distant

A. L'interaction entre le local et le distant

Définition **Remote**

Dans la partie « *Travailler avec un repo distant* », pour pouvoir communiquer avec un dépôt distant, nous avons utilisé la commande `git remote add origin <url>`. Nous voyons donc qu'une *remote* est essentielle pour travailler avec un repo distant. Mais qu'est-ce qu'une remote ? Une remote Git est une référence à un dépôt distant situé sur un serveur ou une plateforme d'hébergement, permettant aux développeurs de collaborer, de partager et de synchroniser leur code avec d'autres membres de l'équipe.

Rappel **L'organisation avec Git**

Pour mieux comprendre comment s'organise le code avec Git, on pourrait prendre un exemple inspiré de la vie de tous les jours.

Imaginez que vous ayez plusieurs ordinateurs dans votre maison et que vous souhaitez les connecter à Internet. Pour cela, vous disposez d'une box fournie par votre Fournisseur d'Accès Internet (FAI). Chaque ordinateur représente un dépôt local, tandis que la box du FAI représente le dépôt distant. Chaque ordinateur a besoin d'une connexion à la box pour accéder à Internet. Cette connexion représente la remote. Grâce à cette connexion, chaque ordinateur peut envoyer et recevoir des données vers et depuis Internet, en utilisant la box comme point d'accès.

Pour résumer, le dépôt local est le code sur la machine du développeur. Le dépôt distant est le code accessible pour tous les développeurs. Quant à elle, la remote est le moyen de connexion entre les dépôts locaux et les dépôts distants.

Méthode **La gestion des remotes**

Généralement, vous vous occupez de créer votre remote lorsque vous commencez votre projet, mais vous avez peu souvent besoin de configurer celles-ci.

Vous pouvez aussi avoir plusieurs remotes. Par exemple, une remote pour envoyer le code vers GitHub, et une autre pour envoyer le code vers votre hébergeur de site web, par exemple Heroku ou Fly.io.

Tous droits réservés à STUDI - Reproduction interdite

Voici comment gérer vos remotes :

- Ajouter une remote :

  `'git remote add <nom_remote> <URL_remote>'` : cette commande ajoute une nouvelle remote avec un nom spécifié et une URL associée. Par exemple, `git remote add origin https://github.com/JohnDoeCourse/MyRepo.git` ajoute une remote appelée « *origin* » liée à l'URL du dépôt distant.

- Afficher la liste des remotes :
  - `'git remote'` : cette commande affiche la liste de toutes les remotes configurées dans votre dépôt local.
  - `'git remote -v'` : cette commande affiche la liste des remotes avec les URL correspondantes.
- Renommer une remote :

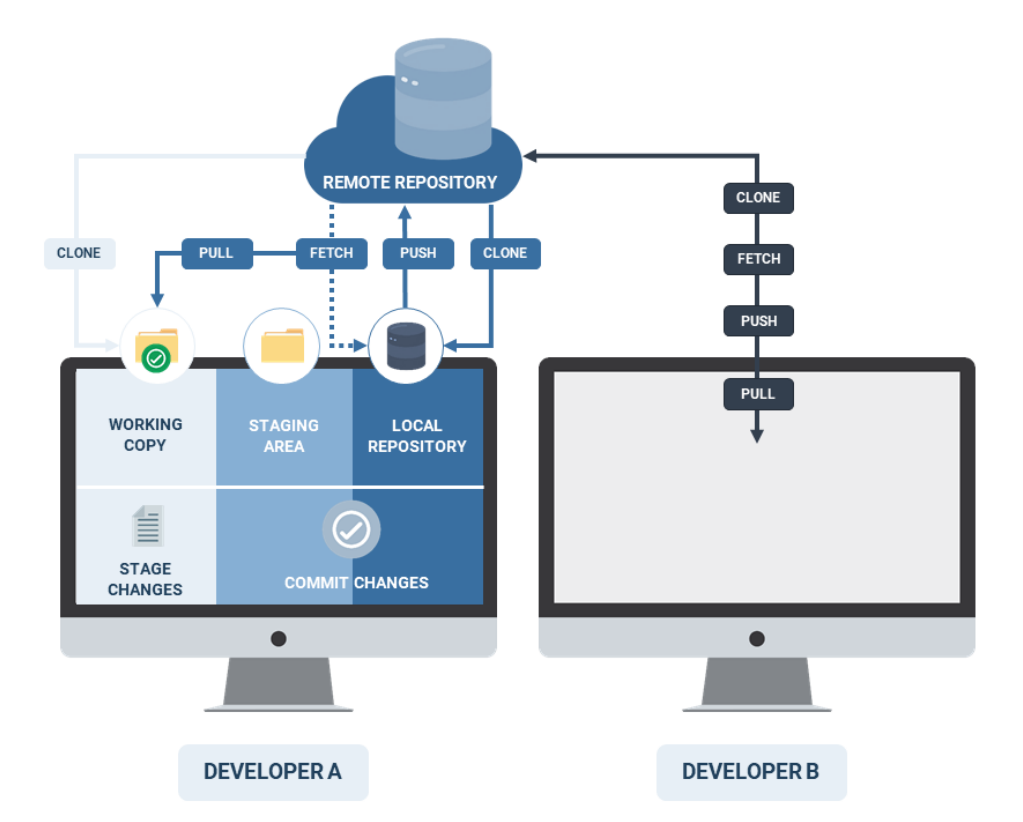
  `'git remote rename <ancien_nom> <nouveau_nom>'` : cette commande permet de renommer une remote existante. Par exemple, `git remote rename origin new-origin` renomme la remote « *origin* » en « *new-origin* ».

- Supprimer une remote :

  `'git remote remove <nom_remote>'` : cette commande supprime la remote spécifiée du dépôt local. Par exemple, `git remote remove origin` supprime la remote « *origin* ».

Définition **La communication entre local et distant**

Lorsque vous travaillez avec un dépôt distant, vous allez faire plusieurs fois par jour devoir synchroniser votre travail local avec le travail distant. Pour cela, vous allez pouvoir utiliser trois commandes Git très connues Fetch, Pull et Push. `'git fetch'` permet de récupérer les modifications du dépôt distant sans les fusionner automatiquement, `'git pull'` récupère et fusionne automatiquement les modifications, tandis que `'git push'` envoie vos commits locaux vers le dépôt distant.



Tous droits réservés à STUDI - Reproduction interdite

Image : Git And Github Version Control10

Méthode **Git fetch**

La commande `git fetch` permet de récupérer les dernières modifications du dépôt distant dans votre dépôt local, sans les fusionner automatiquement avec votre branche courante. Cela vous permet de voir les changements effectués par d'autres contributeurs sans modifier votre travail en cours. Lorsque vous exécutez `git fetch`, Git met à jour les références de branches distantes et télécharge les commits associés, mais n'applique pas les modifications sur vos branches locales. Vous pouvez ensuite utiliser d'autres commandes (comme `git diff` ou `git merge`) pour examiner les modifications récupérées avant de les fusionner.

Même si on l'utilise habituellement sans paramètres, on peut lui spécifier des paramètres pour adapter la commande à nos besoins :

- `'git fetch origin'` : par défaut, `git fetch` récupère les modifications du dépôt distant spécifié par la remote nommée « *origin* ». Cependant, vous pouvez remplacer « *origin* » par le nom de n'importe quelle remote que vous avez configurée dans votre dépôt local.
- `'git fetch --all'` : cette option permet de récupérer les modifications de toutes les remotes configurées dans votre dépôt local, et pas seulement de la remote spécifiée par défaut.
- `'git fetch --prune'` : l'option `--prune` permet de supprimer localement les références de branches distantes qui ont été supprimées dans le dépôt distant. Cela synchronise votre dépôt local avec les changements effectués sur le dépôt distant en supprimant les références locales obsolètes.
- `'git fetch --force'` : l'option `--force` permet d'effectuer une récupération forcée, écrasant toutes les modifications locales et les références de branches avec les dernières modifications du dépôt distant. Cependant, utilisez cette option avec prudence, car elle peut entraîner la perte de données non sauvegardées.
- `'git fetch --depth <n>'` : cette option limite la récupération aux `n` derniers commits du dépôt distant. Cela peut être utile si vous ne souhaitez pas récupérer tout l'historique du projet distant, ce qui peut prendre plus de temps et d'espace de stockage.

Méthode **Git pull**

La commande `git pull` est en fait une combinaison de 2 commandes : `git fetch` et `git merge`. Lorsque vous exécutez `git pull`, Git effectue automatiquement un `git fetch` pour récupérer les modifications du dépôt distant, puis un `git merge` pour fusionner ces modifications dans votre branche courante. Cela permet de mettre à jour votre branche locale avec les dernières modifications du dépôt distant et de fusionner les changements avec votre travail en cours. Cependant, gardez à l'esprit que `git pull` fusionnera automatiquement les modifications et peut entraîner des conflits si des modifications simultanées ont été apportées aux mêmes fichiers.

Tout comme la commande fetch, on utilise généralement pull sans paramètres, mais des paramètres sont disponibles :

- `'git pull origin <branche>'` : par défaut, `git pull` récupère les modifications de la branche spécifiée du dépôt distant nommé « *origin* ». Vous pouvez remplacer « *origin* » par le nom de n'importe quelle remote que vous avez configurée dans votre dépôt local, et `<branche>` par le nom de la branche du dépôt distant que vous souhaitez récupérer.
- `'git pull --rebase'` : l'option `--rebase` permet d'effectuer un rebase plutôt qu'un merge lors de la fusion des modifications récupérées avec votre branche locale. Cela permet de maintenir un historique linéaire et évite la création d'un commit de fusion supplémentaire. Cependant, utilisez cette option avec prudence, car elle peut modifier l'historique des commits.

Tous droits réservés à STUDI - Reproduction interdite

- `'git pull --no-commit'` : l'option `--no-commit` permet de fusionner les modifications récupérées sans créer automatiquement un commit de fusion. Cela vous donne la possibilité d'examiner les modifications et de personnaliser le commit de fusion avant de le valider.

Méthode **Git push**

La commande `git push` est utilisée pour envoyer vos commits locaux vers le dépôt distant correspondant. Lorsque vous exécutez `git push`, Git transfère vos modifications locales vers le dépôt distant spécifié. Cela permet de partager vos changements avec d'autres contributeurs et de mettre à jour le dépôt distant avec vos commits. Il est important de noter que vous devez disposer des autorisations nécessaires pour effectuer un push vers le dépôt distant. Si d'autres contributeurs ont effectué des modifications sur le dépôt distant depuis votre dernière récupération (fetch ou pull), Git vous demandera de récupérer ces modifications avant de pouvoir pousser les vôtres. Cela permet de prévenir les éventuels conflits et de maintenir l'historique du projet cohérent.

Voici quelques paramètres possibles :

- `'git push origin <branche>'` : par défaut, `git push` envoie les commits de la branche courante vers la branche correspondante du dépôt distant nommé « *origin* ». Vous pouvez remplacer « *origin* » par le nom de n'importe quelle remote que vous avez configurée dans votre dépôt local, et `<branche>` par le nom de la branche du dépôt distant vers laquelle vous souhaitez pousser les commits.
- `'git push --all'` : cette option permet de pousser toutes les branches locales vers le dépôt distant correspondant. Cela envoie les commits de toutes les branches locales vers les branches correspondantes sur le dépôt distant.
- `'git push --force'` : l'option `--force` permet d'écraser les commits du dépôt distant avec vos commits locaux, même si cela entraîne la perte de données. Utilisez cette option avec prudence, car elle peut modifier l'historique du dépôt distant et potentiellement causer des problèmes si d'autres contributeurs ont déjà récupéré ces commits.
- `'git push --set-upstream origin <branche>'` : cette option configure la branche locale spécifiée comme une branche de suivi de la branche correspondante du dépôt distant. Cela signifie que vous pouvez utiliser simplement `git push` à l'avenir sans spécifier la remote et la branche, Git saura automatiquement où pousser les commits.

B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Qu'est-ce qu'une remote dans Git ?

- ○ Une référence à un dépôt local situé sur votre machine
- ○ Une référence à un dépôt distant situé sur un serveur ou une plateforme d'hébergement
- ○ Une référence à un dépôt partagé entre plusieurs développeurs

Question 2

Quelle commande permet d'ajouter une remote dans Git ?

- ○ `git remote add <nom_remote> <URL_remote>`
- ○ `git add remote <nom_remote> <URL_remote>`
- ○ `git remote create <nom_remote> <URL_remote>`

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Quelle commande permet de renommer une remote existante dans Git ?

- `git remote rename <nouveau_nom> <ancien_nom>`
- `git rename remote <ancien_nom> <nouveau_nom>`
- `git remote rename <ancien_nom> <nouveau_nom>`

Question 4

Quelle commande permet de supprimer une remote dans Git ?

- `git remote remove <nom_remote>`
- `git remove remote <nom_remote>`
- `git remote delete <nom_remote>`

Question 5

Quelle commande permet de récupérer les modifications du dépôt distant dans votre dépôt local sans les fusionner automatiquement dans Git ?

- `git pull`
- `git push`
- `git fetch`

IV. Essentiel

En conclusion, les interactions avec le dépôt distant Git sont essentielles pour collaborer efficacement et maintenir un historique cohérent des modifications du code. Grâce à l'utilisation des commandes clés telles que `git clone`, `git push`, `git pull` et `git fetch`, vous pouvez interagir de manière fluide avec le dépôt distant et assurer la synchronisation régulière entre votre dépôt local et le dépôt distant.

En premier lieu, la commande `git clone` vous permet de créer une copie locale complète du dépôt distant, incluant tous les fichiers et l'historique des commits. Cela facilite la récupération initiale du code et vous permet de commencer à travailler localement. Ensuite, l'utilisation de `git push` joue un rôle crucial en vous permettant d'envoyer vos modifications locales vers le dépôt distant, mettant ainsi à jour le dépôt avec vos derniers changements. De cette manière, les autres membres de l'équipe peuvent accéder à vos contributions et collaborer de manière efficace. De plus, pour récupérer les changements effectués par d'autres contributeurs, vous pouvez utiliser la commande `git pull`. Cette commande vous permet de récupérer les dernières modifications du dépôt distant et de les fusionner automatiquement avec votre branche locale, assurant ainsi la mise à jour de votre code. Par ailleurs, la commande `git fetch` s'avère utile pour récupérer les dernières modifications du dépôt distant sans les fusionner immédiatement avec votre branche locale. Cela vous offre la possibilité d'inspecter les changements avant de les fusionner, vous permettant ainsi d'avoir un meilleur contrôle sur les modifications apportées.

En utilisant ces commandes, vous pouvez interagir de manière fluide avec le dépôt distant Git, partager vos modifications, récupérer les contributions des autres et maintenir une synchronisation régulière entre votre dépôt local et le dépôt distant. Cela favorise une collaboration efficace et assure la cohérence de l'historique des modifications du code.

V. Auto-évaluation

A. Exercice

Vous devez vous occuper de la mise en place d'un projet pour votre entreprise. Vous devez donc créer le repo sur votre PC, puis sur GitHub avec un certain nombre de fichiers.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.19]

Voici les instructions que vous recevez :

1. Créez le repo Git en local.

2. Ajoutez un fichier « *index.html* » ayant ce contenu :

```html
<!DOCTYPE html>
<html>
<head>
    <title>Mon Site Animé</title>
    <link rel="stylesheet" href="style.css"/>
</head>
<body>
    <h1>Bienvenue sur Mon Site Animé</h1>
    <p>Découvrez une animation simple et élégante.</p>
</body>
</html>
```

3. Faites un commit avec ce fichier ayant pour message « *add index.html file* ».

4. Ajoutez un fichier style.css contenant ceci :

```css
/* CSS pour l'animation */
        @keyframes slideIn {
            0% {
                transform: translateX(-100%);
            }
            100% {
                transform: translateX(0);
            }
        }

        /* CSS pour le titre et la description */
        h1 {
            animation: slideIn 1s ease-out;
        }

        p {
            animation: slideIn 1s ease-out 0.5s;
        }
```

5. Faites un commit avec ce fichier ayant pour message « *add animation style* ».

Question 2

[solution n°4 p.19]

Voici les instructions que vous avez reçues pour le dépôt distant :

1. Créez un repository sur GitHub avec pour nom « *testAnimationWebsite* », en veillant à ajouter un readme.md depuis la page de création de repo.

Vous devez ajouter du contenu au readme.md directement depuis GitHub.

Voici le contenu du readme.md :

```
# Mon Site Animé

Bienvenue sur Mon Site Animé ! C'est une page web très simple avec une jolie animation.

## Comment exécuter le site en local

1. Clonez ce dépôt sur votre machine locale.
2. Ouvrez le fichier `index.html` dans votre navigateur.

```

Tous droits réservés à STUDI - Reproduction interdite

```
## Personnalisation

Si vous souhaitez personnaliser l'animation ou les styles, vous pouvez modifier le fichier `styles.css` pour apporter les changements souhaités.
```

2. Ajoutez la remote à votre dépôt local

3. Récupérez et fusionnez depuis le dépôt distant. Attention, vous aurez une erreur si vous faites un `git pull` classique, il vous faut ajouter l'option `--allow-unrelated-histories` pour accepter de fusionner 2 historiques différents.

4. En local, vous devriez avoir le fichier html, css, et le readme. Maintenant, poussez ces modifications.

B. Test

Exercice 1 : Quiz

[solution n°5 p.20]

Question 1

Lorsque vous utilisez la commande `git push`, vous récupérez les modifications du dépôt distant.

- ○ Vrai
- ○ Faux

Question 2

La commande `git clone` permet de créer une copie locale complète d'un dépôt distant.

- ○ Vrai
- ○ Faux

Question 3

Lorsque vous utilisez la commande `git pull`, vous récupérez les dernières modifications du dépôt distant et fusionnez automatiquement les changements avec votre branche locale.

- ○ Vrai
- ○ Faux

Question 4

On ne peut avoir qu'une seule remote par dépôt.

- ○ Vrai
- ○ Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 9 **Solution n°1**

Question 1

Qu'est-ce qu'un repository distant (dépôt distant) sur GitHub ?

- (●) Une version du dépôt Git hébergée sur un serveur distant
- ( ) Une copie de sauvegarde du dépôt local
- ( ) Un dépôt utilisé uniquement pour le développement local

Un repository distant est une version du dépôt Git hébergée sur un serveur distant, accessible et partageable par plusieurs développeurs.

Question 2

Quelles sont les principales plateformes d'hébergement pour les repositories distants ?

- ( ) Azure, AWS, Google Cloud
- (●) GitHub, GitLab, Bitbucket
- ( ) WordPress, Joomla, Drupal

GitHub, GitLab et Bitbucket sont des exemples populaires de plateformes d'hébergement pour les repositories distants.

Question 3

Comment crée-t-on un compte sur GitHub ?

- (●) En se rendant sur le site et en remplissant un formulaire d'inscription
- ( ) En téléchargeant une application dédiée sur son ordinateur
- ( ) En utilisant une commande spécifique dans le terminal

Pour créer un compte sur GitHub, vous devez vous rendre sur leur site officiel et remplir un formulaire d'inscription.

Question 4

Où peut-on trouver la liste de ses dépôts sur GitHub ?

- [ ] En utilisant la commande `git list` dans le terminal
- [x] En se rendant sur la page d'accueil de son profil
- [x] En cliquant sur son avatar en haut à droite de la page

Vous pouvez trouver la liste de vos dépôts sur GitHub en cliquant sur votre avatar en haut à droite de la page, puis en sélectionnant « *Your repositories* ».

Question 5

Quels sont les onglets disponibles sur la page de détail d'un repository sur GitHub ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Files, Commits, Branches, Merge requests, Issues, Actions, Settings
- ◉ Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings
- ○ Overview, Files, History, Issues, Settings

Les onglets disponibles sur la page de détail d'un repository sur GitHub sont Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights et Settings.

Exercice p. 13 **Solution n°2**

Question 1

Qu'est-ce qu'une remote dans Git ?

- ○ Une référence à un dépôt local situé sur votre machine
- ◉ Une référence à un dépôt distant situé sur un serveur ou une plateforme d'hébergement
- ○ Une référence à un dépôt partagé entre plusieurs développeurs

Une référence à un dépôt distant situé sur un serveur ou une plateforme d'hébergement. Une remote Git est utilisée pour se connecter à un dépôt distant et permet aux développeurs de collaborer, de partager et de synchroniser leur code avec d'autres membres de l'équipe.

Question 2

Quelle commande permet d'ajouter une remote dans Git ?

- ◉ `git remote add <nom_remote> <URL_remote>`
- ○ `git add remote <nom_remote> <URL_remote>`
- ○ `git remote create <nom_remote> <URL_remote>`

`git remote add <nom_remote> <URL_remote>` : cette commande permet d'ajouter une nouvelle remote avec un nom spécifié et une URL associée.

Question 3

Quelle commande permet de renommer une remote existante dans Git ?

- ○ `git remote rename <nouveau_nom> <ancien_nom>`
- ○ `git rename remote <ancien_nom> <nouveau_nom>`
- ◉ `git remote rename <ancien_nom> <nouveau_nom>`

`git remote rename <ancien_nom> <nouveau_nom>` : cette commande permet de renommer une remote existante en spécifiant son ancien nom et son nouveau nom.

Question 4

Quelle commande permet de supprimer une remote dans Git ?

- ◉ `git remote remove <nom_remote>`
- ○ `git remove remote <nom_remote>`
- ○ `git remote delete <nom_remote>`

Tous droits réservés à STUDI - Reproduction interdite

`git remote remove <nom_remote>` : cette commande supprime la remote spécifiée du dépôt local.

Question 5

Quelle commande permet de récupérer les modifications du dépôt distant dans votre dépôt local sans les fusionner automatiquement dans Git ?

- `git pull`
- `git push`
- `git fetch`

La commande `git fetch` permet de récupérer les dernières modifications du dépôt distant dans votre dépôt local sans les fusionner automatiquement avec votre branche courante. Cela vous permet de voir les changements effectués par d'autres contributeurs sans modifier votre travail en cours.

p. 15 **Solution n°3**

Voici les étapes que vous devez suivre :

- Ouvrez votre terminal ou votre interface de ligne de commande.
- Naviguez vers le répertoire où vous souhaitez créer votre dépôt local.

Utilisez la commande suivante pour initialiser un nouveau dépôt Git :

```
1 git init
```

Créez un fichier nommé « *index.html* » et ajoutez le contenu HTML fourni.

Utilisez la commande suivante pour ajouter le fichier à l'index Git :

```
1 git add index.html
```

Effectuez un commit avec le message « *add index.html file* » en utilisant la commande suivante :

```
1 git commit -m "add index.html file"
```

Créez un fichier nommé « *style.css* » et ajoutez le contenu CSS fourni.

Utilisez la commande suivante pour ajouter le fichier à l'index Git :

```
1 git add style.css
```

Effectuez un commit avec le message « *add animation style* » en utilisant la commande suivante :

```
1 git commit -m "add animation style"
```

p. 15 **Solution n°4**

Tous droits réservés à STUDI - Reproduction interdite

Voici les étapes que vous devez suivre :

Accédez à la page de création de repo sur GitHub puis entrez le nom « *testAnimationWebsite* » pour votre repository avant de cocher l'option pour ajouter un README.md. Créez le repository et copiez le contenu fourni du README.md dans le fichier README.md de votre dépôt distant.

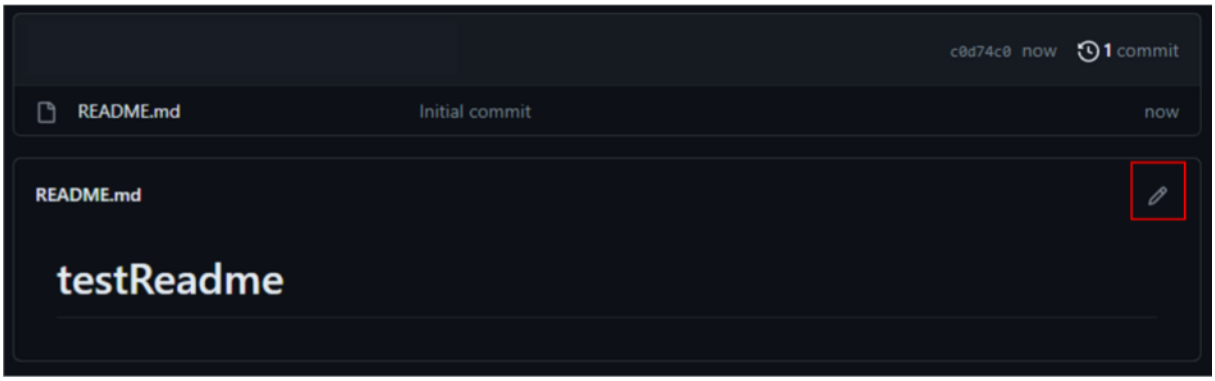


Image : Change readme content

Une fois les modifications apportées, cliquez sur le bouton « ***commit changes*** » puis ajoutez la remote pour votre dépôt distant GitHub en utilisant la commande suivante :

```
1 git remote add origin <URL_DU_DEPOT_DISTANT>
```

Récupérez les modifications du dépôt distant et fusionnez-les avec votre branche locale en utilisant la commande suivante :

```
1 git pull origin main --allow-unrelated-histories
```

Vérifiez que vous avez les fichiers index.html, style.css et README.md en local et poussez vos modifications vers le dépôt distant en utilisant la commande suivante :

```
1 git push origin main
```

Exercice p. 16 **Solution n°5**

Question 1

Lorsque vous utilisez la commande `git push`, vous récupérez les modifications du dépôt distant.

- ○ Vrai
- ◉ Faux

Lorsque vous utilisez la commande `git push`, vous poussez vos modifications locales vers le dépôt distant spécifié. Cela permet de mettre à jour le dépôt distant avec vos derniers changements.

Question 2

La commande `git clone` permet de créer une copie locale complète d'un dépôt distant.

- ◉ Vrai
- ○ Faux

La commande `git clone` permet de créer une copie locale complète d'un dépôt distant, y compris toutes les branches, l'historique des commits et les fichiers. Cela crée une copie exacte du dépôt distant sur votre machine locale.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Lorsque vous utilisez la commande `git pull`, vous récupérez les dernières modifications du dépôt distant et fusionnez automatiquement les changements avec votre branche locale.

- (•) Vrai
- ( ) Faux

La commande `git pull` permet de récupérer les dernières modifications du dépôt distant et de les fusionner automatiquement avec votre branche locale. Cela met à jour votre branche locale avec les changements effectués par d'autres contributeurs sur le dépôt distant.

Question 4

On ne peut avoir qu'une seule remote par dépôt.

- ( ) Vrai
- (•) Faux

On peut paramétrer plusieurs remotes par dépôt, chaque remote a d'ailleurs un nom. La remote de base s'appelle « *origin* ».

Tous droits réservés à STUDI - Reproduction interdite